CP_4 TRANSACTIONS, VIEWS, TRIGGERS, INDEXES

1. Vytvoření a použití triggeru

Trigger před smazáním včelaře s odpovídajícím ID postupně kontroluje všechny záznamy v tabulce včelího úlu a zajišťuje, že není možné smazat včelaře, pokud existuje záznam, kde ID včelaře odpovídá danému ID.

```
CREATE EXTENSION IF NOT EXISTS plpgsql;

CREATE FUNCTION check_vcelar()

    RETURNS TRIGGER
AS
$$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM včelí_ul
        WHERE včelí_ul.včelař = OLD.id)
    THEN
        RAISE EXCEPTION 'To nejde. Včelař má na starosti alespon jeden úl.';
    END IF;
    RETURN OLD;
END;
$$
    LANGUAGE plpgsql;

CREATE TRIGGER prevent_delete_vcelar
    BEFORE DELETE ON včelař
    FOR EACH ROW
EXECUTE FUNCTION check_vcelar();
```

2. Transakce a sada dotazů

Do tabulky včelařů dodejme například včelaře s id 16, a přirazime mu úl s ul_id = 3

```
insert into včelař(id, url) VALUES (16, 'www.best_beekeeper.com');

insert into včelí_ul(úl_id, počet_rámečků, složení, včelař) VALUES (3, 7, 'Avocado', 16);
```

Pokusíme-li se teď odstranit včelaře z tabulky Včelař, vyvolá se trigger:

```
DELETE FROM včelař
WHERE id = (
    SELECT id FROM včelař
    WHERE id = 16);
```

```
DELETE FROM včelař
WHERE id = (
    SELECT id FROM včelař
    WHERE id = 16);
```

```
[P0001] ERROR: To nejde. Včelař má na starosti alespon jeden úl.
Where: PL/pgSQL function check_vcelar() line 8 at RAISE
```

Takže provedeme tranzakci

Určení izolace pro tuto transakci je nastaveno na Serializable.

Cílem je odstranit uživatele s id = 16 z tabulky Včelař, protože jeho role se změní na zakaznika. Nejprve v tabulce Včeli_ul přiřadím nového včelaře s id = 14 k úlu s id = 3, kterého původně hlídal včelař s id = 16.
Pak přesuneme uživatele s ID = 16 z tabulky včelařů do tabulky zakaznik. Pak již můžeme tohoto včelaře odstranit z tabulky včelařů, protože nenese odpovědnost za žádný úl a jeho údaje jsou v tabulce kupujícího.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

UPDATE včelí_ul
SET včelař = 14
WHERE úl_id = 3;

INSERT INTO zakaznik(id)
SELECT
    id FROM včelař
WHERE id = 16;

DELETE FROM včelař
WHERE id = (
    SELECT id FROM včelař
    WHERE id = 16
);
COMMIT;
```

```
grechsof.public> COMMIT
[2024-05-16 01:36:47] completed in 8 ms
```

Takto vypadají tabulky po spuštění tohoto kódu:

<u>-------včelař</u>

id	url	ma_spolupracující
13	www.jennifer_white_beekeeper.com	<null>
14	www.matthew_wright_beekeeper.com	15
15	www.samantha_carter_beekeeper.com	14

------zakaznik

	id
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	16

------včeli_úl

	úl_id	počet_rámečků	složení	včelař
1	1	7	Avocado	14
2	2	7	Avocado	14
3	3	7	Avocado	14
4	12	8	Lavender	14
5	23	1	Sage	14
6	27	3	Clover	13
7	28	9	Wildflower	14
8	32	9	Avocado	15
9	34	6	Blueberry	13
10	41	8	Sunflower	13
11	44	10	Orange Blossom	15

3. Vytvoření a použití pohledu

Toto zobrazení sklenice_search vybírá pouze záznamy z tabulky sklenice, kde hodnota ve sloupci "složení" je "Blueberry" a hodnota ve sloupci "množství" je větší nebo rovna 200.
Toto zobrazení lze použít pro rychlý přístup k datům, která odpovídají vašim hledacím kritériím, aniž byste museli opakovaně zadávat dotaz každým spuštěním.

```
CREATE VIEW sklenice_search AS
SELECT idsklenice, složení, množství
FROM sklenice
WHERE složení = 'Blueberry' AND množství >= 200;

SELECT * FROM sklenice_search;
```

	idsklenice	složení	množství
1	14	Blueberry	220

4. Vytvoření a použití indexu

To je příklad EXPLAIN ANALYZE pro vyhledání sklenice medu s pohanky:

```
EXPLAIN ANALYZE SELECT * FROM sklenice
WHERE složení = 'Buckwheat'
```

```
QUERY PLAN
Seq Scan on sklenice  (cost=0.00..1.26 rows=1 width=524) (actual time=0.072..0.077 rows=1 loops=1)
  Filter: (("složení")::text = 'Buckwheat'::text)
  Rows Removed by Filter: 20
Planning Time: 1.014 ms
Execution Time: 0.175 ms
```

Vytvořila jsem B-stromový index pro vyhledávání sklenice medu s pohanky:

```
CREATE INDEX z_ceho_med ON sklenice(složení)
```

```
QUERY PLAN
Seq Scan on sklenice  (cost=0.00..1.26 rows=1 width=524) (actual time=0.021..0.026 rows=1 loops=1)
  Filter: (("složení")::text = 'Buckwheat'::text)
  Rows Removed by Filter: 20
Planning Time: 0.106 ms
Execution Time: 0.051 ms
```

Stejný dotaz ale s použitím indexu pro složení bude mít následující výsledek, vidíme, že proběhlo to rychleji: